

THE BUILDER'S PATH

A Top-Down Microlearning Curriculum
for Artificial Intelligence

Taught through Igbo Civilization, Nature,
Animal Behavior & Psychology

15 Topics • 6 Phases • The 80/20 Path

Umu■ma — Children of Wisdom

Table of Contents

PHASE 1: The AI Map (Foundation)

Topic 1 — AI vs ML vs DL vs LLM: The Definitions

Topic 2 — The Evolution: Rules → ML → Neural Nets → Transformers → Agents

PHASE 2: The Learning Machine (How It Learns)

Topic 3 — Data, Features & Models: The Raw Materials

Topic 4 — Loss Functions & Optimization: The Blacksmith's Fire & Hammer

Topic 5 — Neural Nets & Backpropagation: How the Brain Rewires

PHASE 3: The LLM Brain (Modern AI)

Topic 6 — Tokens & Embeddings: Chopping Words into Village Proverbs

Topic 7 — The Attention Mechanism: Context & Focus

Topic 8 — Context Window, Pre-training & Fine-tuning: School vs Job Training

PHASE 4: Prompt Engineering (Using Intelligence)

Topic 9 — Roles, Context & Constraints: Instructing the Apprentice

Topic 10 — Breaking Problems Down & Prompt Chains: Step-by-Step Thinking

PHASE 5: AI Applications (Building Systems)

Topic 11 — APIs & Python Basics: Digital Messengers

Topic 12 — RAG & Vector Databases: The Yam Barn Memory System

Topic 13 — Agents, Tools & Automation: Giving the AI Hands

PHASE 6: Deeper Understanding (The Roots)

Topic 14 — Linear Algebra & Calculus Concepts: The Hidden Physics

Topic 15 — Transformer Architecture Deep-Dive

PHASE 1: The AI Map (Foundation)

Topic 1 — AI vs ML vs DL vs LLM

Welcome, Builder. Before we touch a single line of code, we must draw the map. Many students drown because they confuse the forest for the trees. Let us settle this once and for all.

The Village Council Analogy (Igbo Civilization)

Imagine the great Igbo village of **Umu■ma** — Children of Wisdom. In this village, intelligence flows through four concentric circles of power, each nested inside the last like Russian dolls carved from iroko wood.

AI — The Village Council (Nd■ Ichie): The broadest ambition. The entire council of elders who can settle disputes, interpret omens, and make wise decisions. Any system that mimics human reasoning — a chess engine, a thermostat, a traffic controller, a chatbot — all of it is AI. The term was coined in 1956 at Dartmouth. AI is the *goal*, the umbrella, the dream.

ML — The Apprenticeship System (■gba Boi): A subset of AI. Instead of the elders writing explicit rules for every situation ('if you see a stranger at dusk, ring the bell'), they let the apprentice *learn patterns from experience*. Show the boy 1,000 yam harvests, and he learns to predict which soil yields the best — without anyone telling him the rule. ML is AI that learns from **data**, not from hand-crafted instructions.

DL — The Ekpe (Leopard) Society: A subset of ML. A secret inner circle that works in *layers*. The first layer detects edges on the leopard's spots, the second detects ears and eyes, the third recognizes a face. Each layer builds on the one before it — this is the **deep** in Deep Learning. These are neural networks with many hidden layers, capable of learning incredibly complex patterns: images, speech, language.

LLM — The Court Poet (■kà Ak■k■): A subset of DL. A specialized deep learning model trained on *text* — the entire oral history of the village, all the proverbs, all the folktales. It learns the patterns of language so deeply that it can continue a story, answer questions, translate between dialects, and even compose new proverbs. GPT, Claude, Gemini — these are all LLMs. They are the court poets who have memorized the entire library but have never stepped outside the village.

■ **Ilu Igbo:** "Onye amaghi ebe mmiri bidoro mawa ya, agaghi ama ebe o kwusiri. (One who does not know where the rain began to beat them cannot know where it stopped.)"

The Nested Matryoshka View

AI $\supset$ ML $\supset$ DL $\supset$ LLM

AI is the entire village. ML is the apprenticeship system. DL is the Leopard Society's layered

secrets. LLMs are the court poets who speak the language.

A Quick Example

- **AI:** A program that translates Igbo to English (could be rules).
- **ML:** A translation system that learns from 100,000 sentence pairs.
- **DL:** A neural network with 12 layers that learns translation patterns.
- **LLM:** Claude/GPT — a massive DL model trained on the entire internet's text.

✍ **TEST: A self-driving car that uses hand-coded 'if road is wet, reduce speed by 15%' rules is AI, but is it ML? Why or why not?**

✓ **ANSWER: No, it is not ML.** ML requires the system to *learn* the rule from data (e.g., watching 10,000 hours of driving in rain) rather than having a human program the rule explicitly. The hand-coded rule is 'Good Old-Fashioned AI' (GOFAI) — symbolic, rule-based AI, not machine learning.

Topic 2 — The Evolution: Rules → ML → Neural Nets → Transformers → Agents

Every great civilization evolves. The Igbo didn't jump from mud huts to skyscrapers overnight. AI followed a similar path — each generation was a response to the failures of the one before it.

Stage 1: Rule-Based Systems — The Elders' Law Book (1950s-1980s)

The first approach was simple: write down **every rule**. If the village council wrote a 10,000-page book covering every possible dispute, they wouldn't need wisdom — they'd just look up the answer. This is **symbolic AI** or **GOFAI** (Good Old-Fashioned AI).

Examples: Chess engines (Deep Blue), expert systems for medical diagnosis, tax preparation software.

Why it failed: The world has too many corner cases. You cannot write a rule for every possible facial expression, every dialect of Igbo, every pothole on every road. The rulebook became infinite.

Stage 2: Machine Learning — The Apprentice (1990s-2010s)

Instead of writing rules, we **show examples** and let the system find the patterns. Show it 10,000 labeled images of 'this is a leopard' and 'this is not a leopard,' and it learns to distinguish. This was a revolution — the system now handled cases the programmers never anticipated.

Examples: Spam filters, recommendation engines, credit card fraud detection.

Stage 3: Neural Networks & Deep Learning — The Leopard Society (2012+)

ML worked, but for simple patterns. When we tried to recognize faces, understand speech, or translate languages, traditional ML hit a wall. Neural networks — inspired by the brain's interconnected neurons — stacked in *layers* changed everything. The 2012 ImageNet competition was the turning point: a deep neural net (AlexNet) crushed every traditional ML approach.

Key enabler: GPUs (graphics cards), originally built for video games, happened to be perfect for the matrix math that neural nets need.

Stage 4: Transformers — The Council of Attention (2017+)

Neural nets were great at images and speech, but **language** remained hard. Words depend on context — 'the bank' means something different in 'river bank' vs 'money bank.' Old models (RNNs, LSTMs) read one word at a time and forgot the beginning by the time they reached the end — like an elder who forgets the crime while listening to the testimony.

The **Transformer** (Google, 2017 — 'Attention Is All You Need') solved this by letting every word **attend to every other word simultaneously**. Imagine a council where every elder can hear every other elder at once, weighing who is most relevant to the current discussion. This parallel attention

mechanism birthed GPT, BERT, Claude, and every modern LLM.

Stage 5: Agents — The Autonomous Village (2024+)

LLMs can talk, but they can't *act*. Agents change that. An agent is an LLM given **tools** — the ability to browse the web, run code, send emails, control software. It's like giving the court poet *hands*. Now they don't just compose proverbs — they can plant yams, build barns, and negotiate trade routes. This is where we are now, Builder. This is the frontier.

The 7-Framework Questions — From Rules to Transformers

Q1: What problem existed before ML?

A1: We had to hand-write rules for every situation. Chess engines needed every possible board position coded. Language translation needed grammar rules for every language pair. The rulebook was infinite — you can't enumerate all possible sentences.

Q2: Why did rule-based systems fail?

A2: Combinatorial explosion. Language, vision, and reasoning have too many possibilities. A 10-word English sentence has more possible meanings than atoms in the universe. No rulebook can cover them all. Also, rules don't handle ambiguity: 'I saw a man on a hill with a telescope' — who has the telescope?

Q3: What design decision fixed it?

A3: Learning from data instead of programming rules. Show the system thousands of examples and let it discover the patterns. This shifts the burden from 'write every rule' to 'collect enough examples' — a much easier problem.

Q4: What sacrifice/trade-off did this introduce?

A4: We traded **interpretability** for **flexibility**. A rule-based system always explains itself ('I did X because rule Y'). A neural network is a black box — even its creators don't fully understand why it made a specific decision. In a village court, this is terrifying: 'Why did you convict him?' 'The patterns suggested it.'

Q5: Where are the break points?

A5: 1) **Data hunger:** ML needs massive data — a village with no written records struggles. 2) **Bias amplification:** If historical village rulings favored titled men, the ML system learns that bias. 3) **Catastrophic forgetting:** Old neural networks forget previous tasks when learning new ones. 4) **Adversarial attacks:** Tiny, invisible changes to an image can fool a neural net completely.

Q6: What hidden assumptions does this rely on?

A6: 1) **The past predicts the future:** Training data represents all future cases. 2) **Correlation = causation:** The system finds patterns, not reasons. 3) **Abundant compute:** Requires massive GPU farms that only large companies afford. 4) **Clean, labeled data:** Real-world data is messy and unlabeled.

Q7: What evolved next to fix those break points?

A7: 1) **Transformers** solved the sequential bottleneck of RNNs. 2) **Transfer learning** (pre-training + fine-tuning) reduced data hunger. 3) **RLHF** (Reinforcement Learning from Human Feedback) addressed bias. 4) **Interpretability research** is slowly opening the black box. 5) **Agents with tools** are making LLMs capable of action, not just speech.

🔪 **TEST: An RNN processes a 500-word Igbo folktale word by word. By word 400, it has 'forgotten' the character introduced at word 20. What did the Transformer invent to fix this problem?**

✓ **ANSWER: The Attention Mechanism.** Instead of processing words sequentially and relying on a fading memory, the Transformer lets every word attend to every other word simultaneously. Word 400 can 'look back' directly at word 20 with the same clarity as word 21. It's like replacing a single elder with fading memory with a council where every elder can hear every testimony at once.

PHASE 2: The Learning Machine (How It Learns)

Topic 3 — Data, Features & Models: The Raw Materials

Before the blacksmith strikes the iron, he must have iron. Before the weaver creates the cloth, she must have thread. So it is with machine learning.

Data — The Yam Harvest (Mkp■r■ Ji)

In Igbo culture, the yam is king. A man's wealth is measured by his yam barn (■ba Ji). In ML, **data** is the yam harvest. Without data, you have nothing to learn from. The richer your harvest, the more patterns you can discover.

Structured Data: Like yams sorted by size in neat rows — spreadsheets, tables, databases. Each column is a known attribute.

Unstructured Data: Like a forest of wild yams — images, text, audio, video. No neat columns, but rich with information.

Features — The Yam Inspector's Checklist (Njirimara)

A yam inspector doesn't look at the *entire* yam. They check specific attributes: size, color, firmness, presence of spots. These measurable attributes are **features** — the individual signals the model uses to make decisions.

Feature Engineering: The art of choosing and transforming raw data into useful features. A master yam inspector knows that 'weight-to-circumference ratio' matters more than 'color' for predicting rot. This skill, not raw computing power, was the difference between winning and losing in traditional ML competitions.

Model — The Decision Framework (Iwu)

After inspecting thousands of yams (data) using specific attributes (features), the apprentice develops an *internal sense* of what makes a good yam. This internal framework is the **model** — a mathematical structure that maps inputs (features) to outputs (predictions).

Training: Showing the apprentice labeled yams ('this one rotted, this one didn't') until their internal model is accurate.

Inference: The apprentice inspects a new, unseen yam and predicts its fate.

■ *Ilu Igbo: "E ji ■b■chi taa ak■wa ■b■chi echi. (We use today to explain tomorrow.)"*

✍ **TEST:** You're building a model to predict which village yam barns will collapse in the next rainy season. Name 3 data points you'd collect and 2 features you'd engineer from that data.

✓ **ANSWER: Data:** Barn age, roof material type, number of past repairs, yam count (weight load), soil moisture readings near foundation.

Features: 'Load per square meter' ($\text{yam count} \div \text{barn floor area}$), 'Moisture trend' (soil moisture this month vs 3-month average).

Topic 4 — Loss Functions & Optimization: The Blacksmith's Fire & Hammer

The blacksmith (kp z) does not strike blindly. He heats the iron in fire, strikes it with his hammer, checks the shape, and strikes again. Each strike moves the metal closer to the desired form. Machine learning is the same process, repeated thousands of times per second.

The Loss Function — Measuring Error (he j)

After each hammer strike, the blacksmith looks at the metal and asks: *'How far am I from the desired shape?'* This gap between current and desired is the **loss** (or error). The **loss function** is the mathematical formula that measures this gap.

Mean Squared Error (MSE): For predicting numbers (yam weight). Squaring penalizes large errors much more than small ones — like a chief who gives one warning for lateness but exile for treason.

Cross-Entropy Loss: For classification (good yam vs bad yam). Measures how 'surprised' the model is by the correct answer. A confident wrong answer generates massive loss — like the shame of an Ozo titled man caught lying.

Optimization — The Striking Pattern (k z)

Knowing the error is useless without a plan to reduce it. The **optimizer** is the blacksmith's striking pattern — which direction to swing, how hard, how many times.

Gradient Descent: Imagine you're blindfolded on a hill and need to find the lowest valley. You feel the ground with your foot in every direction, find the steepest downward slope, and take a small step. Repeat. That's gradient descent — calculating which direction reduces the loss fastest, then taking a step in that direction.

Learning Rate: How big each step is. Too big: you leap over the valley entirely (the training never converges). Too small: you take 10,000 years to reach the bottom. The blacksmith's art is knowing how hard to strike — tap a nail, slam a machete blank.

Stochastic Gradient Descent (SGD): Instead of feeling the entire hillside (all data), you feel just one small patch (one data point or batch) and take a step. Faster, noisier, but surprisingly effective — like a blacksmith who checks every 10th blade instead of inspecting the entire batch.

The Analogy Complete

Loss Function = Eye: 'The blade is 3cm too long.'

Optimizer = Arm: 'I will grind 3cm off the tip.'

Learning Rate = Force: How hard to press against the grindstone.

Iteration = Each Strike: Eye → Arm → Strike → Eye → Arm → Strike...

✍ **TEST:** Your loss is dropping smoothly from 5.0 to 0.5 to 0.05 over 100 training steps. Then suddenly it spikes to 999. What is the most likely cause, using the blacksmith analogy?

✓ **ANSWER: Learning rate too high.** The blacksmith swung so hard (large step) that the blade flew off the anvil entirely. In gradient descent, a large learning rate can cause the model to 'overshoot' the minimum and land in a region of extremely high loss — numerical instability, exploding gradients.

Topic 5 — Neural Nets & Backpropagation: How the Brain Rewires

The human brain has 86 billion neurons, each connected to thousands of others. When you learn that fire burns, a physical connection strengthens between 'fire' neurons and 'pain' neurons. Artificial neural networks mimic this — but with math instead of biology.

The Neuron — A Village Elder (Onye Ichie)

A single neuron is like a village elder who:

1. **Listens** to many advisors (inputs from other neurons)
2. **Weighs** each advisor's opinion differently (weights)
3. **Sums** all weighted advice
4. **Decides** whether to speak up (activation function)

Mathematically: $\text{output} = \text{activation}(w_1x_1 + w_2x_2 + \dots + w_nx_n + \text{bias})$

The **bias** is the elder's natural tendency to agree or disagree — their baseline disposition, independent of any advisor.

Layers — The Village Hierarchy

Input Layer: The village messengers who bring news from outside (raw data: pixels, words, numbers).

Hidden Layers: The council of elders, sitting in concentric circles. Each circle processes information from the circle before it, extracting increasingly abstract patterns. Layer 1 detects edges → Layer 2 detects shapes → Layer 3 detects objects → Layer 4 identifies the object.

Output Layer: The Igwe (King) who delivers the final verdict: 'This is a leopard' (0.87 confidence) or 'This is a dog' (0.13 confidence).

Backpropagation — Learning from Mistakes (Nlaghachi Azu)

When the king makes a wrong judgment ('That was a dog, not a leopard'), the error must flow *backwards* through the entire council. Each elder adjusts how much they trusted each advisor who led them astray. This backward flow of error correction is **backpropagation** — the single most important algorithm in deep learning.

The Chain Rule (Calculus): The mathematical engine of backpropagation. It answers: 'If I change this weight slightly, how much does the final error change?' It chains together all the small effects through every layer, from output back to input. The Igbo proverb captures it: '*Mmadu anaghị echeta ebe ọ malitere ịda, ọ na-echeta ebe ọ dara.*' (A person doesn't remember where they started falling, they remember where they landed.) — but backpropagation remembers every step.

Activation Functions — The Elder's Temperament

Without activation functions, a neural network is just linear algebra — it can only draw straight lines. Activation functions introduce **non-linearity**, allowing the network to learn curves, bends, and complex boundaries.

ReLU (Rectified Linear Unit): 'If the advice is positive, I'll pass it on. If negative, I stay silent.' — The grumpy but efficient elder. Most common.

Sigmoid: 'I'll give you a probability between 0 and 1.' — The cautious elder who never commits fully.

Softmax: 'Among all the options, here is the probability distribution.' — The fair elder who divides resources proportionally.

🔪 **TEST: If you removed all activation functions from a 100-layer neural network and replaced them with identity ($f(x) = x$), what would you actually have?**

✓ **ANSWER: A single linear transformation.** Without non-linearity, 100 layers of matrix multiplication collapse into one big matrix multiplication. The network loses all its depth — it can only draw straight lines. You've built a 100-story building where every floor is perfectly transparent glass; you might as well be on the ground floor.

PHASE 3: The LLM Brain (Modern AI)

Topic 6 — Tokens & Embeddings: Chopping Words into Village Proverbs

Computers don't understand words — they understand numbers. Before an LLM can process language, words must be converted into mathematics. This happens in two stages: **tokenization** and **embedding**.

Tokenization — The Kolanut Breaker (■gwa ■j■)

In Igbo tradition, the kolanut is broken into pieces before it's shared among elders. So too must language be broken into digestible pieces before the model can consume it.

A **token** is roughly $\frac{3}{4}$ of a word in English, or about 4 characters. 'The leopard' might be split into ['The', 'leopard']. 'Unbelievable' might become ['Un', 'believ', 'able']. Common words get their own token; rare words get split into subwords.

Why subwords? Igbo has compound words like '■k■k■' (chicken). A subword tokenizer can handle any word it's never seen by breaking it into known pieces — like an elder who can understand a new proverb by recognizing its component proverbs. The vocabulary size (e.g., 50,000 tokens) is fixed; everything else is built from these atoms.

Embeddings — The Village Map of Meaning (Mapa Nk■wa)

After breaking the kolanut, we need to place each piece on a **map of meaning**. An embedding is a list of numbers (a vector) that represents where a word lives in 'meaning space.'

Imagine a village where every word is a house. Words with similar meanings live on the same street: 'king,' 'chief,' and 'elder' are neighbors. 'Yam,' 'cassava,' and 'cocoyam' share a neighborhood. 'Murder' and 'celebration' live on opposite sides of the village.

Vector arithmetic: The magic of embeddings. 'King' – 'Man' + 'Woman' $\approx$ 'Queen'. 'Paris' – 'France' + 'Nigeria' $\approx$ 'Abuja'. These relationships emerge purely from seeing words in context — the model never studied geography, but it learned that capitals and countries have a consistent relationship.

Dimensions: Typical embeddings have 768, 1024, or 4096 dimensions. Each dimension captures some latent aspect of meaning — formality, gender, animacy, size, danger level — but they're not human-labeled; the model discovers them organically.

■ *Ilụ Igbo: "Okwu na-eri okwu. (Words eat words — one word leads to another.)"*

✍ **TEST:** Tokenize the Igbo word 'onyeisioche' (leader) into subwords, assuming the tokenizer knows 'onye,' 'isi,' and 'oche' as individual tokens. What's the advantage of subword tokenization over word-level tokenization?

✓ **ANSWER:** Tokens: ['onye', 'isi', 'oche']. **Advantage:** The tokenizer can handle any word, even ones it's never seen, by composing known subwords. A word-level tokenizer would mark 'onyeisioche' as [UNKNOWN] if it wasn't in the vocabulary. This is crucial for morphologically rich languages like Igbo, where agglutination creates countless compound words.

Topic 7 — The Attention Mechanism: Context & Focus

This is the single most important idea in modern AI. Everything — GPT, Claude, Gemini, Llama — rests on this mechanism. If you understand nothing else, understand this.

The Problem: Ambiguity Is Everywhere

Consider the Igbo word 'akwkw'. It can mean: book, leaf, paper, education, or school — depending on context.

'na-ag akwkw' — 'He is reading a book' (or studying? or reading a leaf?)

'na-ata akwkw' — 'He is chewing a leaf' (now it's clearly 'leaf')

The word 'akwkw' is identical in both sentences. Only the surrounding verb ('ag' = read vs 'ata' = chew) disambiguates it. **Context** determines meaning. The attention mechanism is how LLMs capture context.

The Council Meeting Analogy (gbak Nd Ichie)

Picture the council of elders discussing a dispute. The elder speaking says the word 'akwkw'. Immediately, every elder in the room:

1. **Looks at** the word 'ag' (read) spoken 3 words earlier — *high attention*
2. **Glances at** the word 'n'ah'a' (market) spoken 12 words earlier — *medium attention*
3. **Ignores** the word 'iit' (morning) spoken 50 words earlier — *near-zero attention*

Each elder assigns an **attention score** to every other word in the room. These scores determine how much each context word influences the meaning of 'akwkw'. High attention on 'ag' → the meaning shifts toward 'book/education.' High attention on 'osisi' (tree) → the meaning shifts toward 'leaf.'

Q, K, V — The Three Roles

Every word plays three roles simultaneously, like an elder who is simultaneously a questioner, an answerer, and a knowledge holder:

Query (Q) — 'What am I looking for?' Each word asks: 'Which other words are relevant to my meaning?' The word 'akwkw' queries: 'Who here clarifies what I am?'

Key (K) — 'What do I offer?' Each word advertises: 'Here's what I can tell you about myself.' The word 'ag' advertises: 'I am a reading-related verb.'

Value (V) — 'Here is my actual content.' Each word holds its actual information. When attention is paid, the Value is what gets transferred.

The magic: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/d)V$ — The Query and Key compute compatibility scores (how well do we match?), softmax normalizes them into attention weights (percentages),

and these weights are applied to the Values.

Multi-Head Attention — Many Elders, Many Perspectives

One elder attending to everything captures one perspective. But what if we need multiple? **Multi-head attention** runs several attention mechanisms in parallel — each 'head' learns a different relationship type:

- **Head 1:** Syntax — tracks subject-verb agreement
- **Head 2:** Semantics — tracks what objects are being discussed
- **Head 3:** Coreference — tracks that 'he' refers to 'Okonkwo'
- **Head 4:** Sentiment — tracks emotional tone

It's like having a council where one elder tracks legal precedent, another tracks family lineage, another tracks market prices, and another reads body language — all simultaneously, all feeding into the final decision.

The 7-Framework Questions — The Attention Revolution

Q1: What problem existed before attention?

A1: Sequential processing (RNNs/LSTMs) read text one word at a time, carrying a 'hidden state' forward. By the time they reached word 500, the hidden state had degraded — like a message whispered through 500 people.

Q2: Why did sequential models fail?

A2: 1) Vanishing gradients: the signal from early words disappeared during training. 2) No parallelization: word 500 cannot be processed until words 1-499 are done. 3) Long-range dependencies: 'The woman who bought the yams that were grown in the village near the river... arrived' — by 'arrived,' RNNs had forgotten 'the woman.'

Q3: What design decision fixed it?

A3: Attention lets every word directly access every other word in $O(1)$ steps — not $O(n)$. The Transformer processes ALL words simultaneously (parallel) while maintaining perfect long-range context. 'Attention Is All You Need' — the paper that dropped recurrence entirely.

Q4: What sacrifice/trade-off?

A4: Quadratic complexity: $O(n^2)$. A 1,000-word text requires 1,000,000 attention calculations. A 100,000-word book requires 10 billion. This is why context windows are expensive and why companies race to extend them.

Q5: Where are the break points?

A5: 1) **Quadratic cost** hits hard at scale. 2) **Position blindness:** attention sees words as a 'bag' — it doesn't naturally know word order (solved by positional encoding). 3) **Attention sink:** early tokens absorb disproportionate attention.

Q6: What hidden assumptions?

A6: 1) Every word is equally accessible to every other word (not how humans read). 2) Fixed-dimensional embeddings can capture all meaning. 3) The softmax normalization (must sum to 1.0) is the right way to distribute attention.

Q7: What evolved next?

A7: 1) **Flash Attention** — hardware-aware optimization, makes $O(n^2)$ practical. 2) **Sparse attention** — only attend to important tokens. 3) **Sliding window attention** — local context + occasional global. 4) **State-space models (Mamba, etc.)** — linear-time alternatives to attention.

🔪 **TEST:** In the sentence 'The elders blessed the kolanut and it was shared among the villagers,' what should 'it' attend to most heavily — and what would happen if 'it' attended equally to 'elders' and 'kolanut'?

✓ **ANSWER:** 'It' should attend most heavily to 'kolanut' (the object being shared). If it attended equally to 'elders,' the model might generate 'the elders were shared among the villagers' — a cannibalistic and physically impossible conclusion. Attention disambiguation is what separates coherent text from nonsense.

Topic 8 — Context Window, Pre-training & Fine-tuning: School vs Job Training

An LLM's life has three stages, like an Igbo child's journey: childhood education (pre-training), apprenticeship (fine-tuning), and adult collaboration (inference with context).

Context Window — The Elder's Working Memory (Icheta)

When the council meets, they can only hold so much information in their minds at once. This is the **context window** — the maximum number of tokens the model can 'see' at any moment. Early models: 512 tokens. GPT-4 Turbo: 128K. Gemini 2.0: 1M+. Claude: 200K.

Think of it like the village square: only so many elders can fit in the circle at once. Everything outside the circle is invisible. If an elder mentions something discussed 3 hours ago that's no longer in the circle, the model has no memory of it — unless it's been saved externally (RAG, memory systems).

Pre-training — The Village School (■|■ Akw■kw■ Obodo)

Before an apprentice starts their specific trade, they attend the village school where they learn *everything*: history, proverbs, arithmetic, ethics, folktales. Similarly, an LLM is pre-trained on **trillions of tokens** from the internet — books, articles, code, conversations, Wikipedia.

Task: Next-token prediction. 'The leopard hunted the...' → 'antelope.' Repeat this trillions of times. The model learns grammar, facts, reasoning patterns, and world knowledge — purely from predicting what comes next.

Cost: Millions of dollars in GPU compute. GPT-4's training run was estimated at \$100M+. This is why only a handful of companies build foundation models.

Result: A **base model** — incredibly knowledgeable but raw. It completes text but doesn't follow instructions. It might complete 'How do I build a bomb?' with actual instructions, because it was trained to predict text, not to be helpful or safe.

Fine-tuning — The Apprenticeship (■gba Boi Nke Ab■■)

After village school, the apprentice enters a specific trade: blacksmithing, weaving, or divination. This is **fine-tuning** — taking the pre-trained model and training it further on a smaller, specialized dataset.

Instruction Tuning: Training on question-answer pairs and instruction-following examples. This turns the base model from a 'text completer' into an 'assistant.' It learns: 'When someone asks a question, answer it — don't just complete it.'

RLHF (Reinforcement Learning from Human Feedback): Humans rank the model's responses ('this answer is better than that one'), and the model learns to produce the preferred style. This is what makes Claude polite, GPT helpful, and models generally safe. Like an elder correcting an apprentice: 'No, child — that answer is technically correct but disrespectful. Try again.'

Domain Fine-tuning: Training further on legal documents to create a legal assistant, or on medical texts for a medical chatbot. The model retains its general knowledge but specializes.

The Three Worlds Compared

	Pre-training	Fine-tuning	Context Window
What	Learn everything	Learn a specialty	Working memory
Data size	Trillions of tokens	Thousands to millions	Thousands of tokens
Cost	\$\$\$ Millions	\$ Hundreds	Free (per request)
Result	Base model	Assistant / Specialist	Current conversation
Analogy	Village school	Apprenticeship	Council meeting

✂ **TEST:** A company fine-tunes GPT-4 on their internal HR documents to answer employee policy questions. After 3 months, the model starts giving wrong answers about basic world facts (e.g., 'Paris is the capital of Germany'). What ML phenomenon is this?

✓ **ANSWER: Catastrophic forgetting** (or catastrophic interference). During fine-tuning, the model over-adapted to HR documents and 'overwrote' some of its general pre-trained knowledge. Like an apprentice who learned blacksmithing so intensively they forgot how to weave — even though weaving was their childhood skill. Solutions include: weight regularization, elastic weight consolidation, or mixing general data with the fine-tuning data.

PHASE 4: Prompt Engineering (Using Intelligence)

Topic 9 — Roles, Context & Constraints: Instructing the Apprentice

A master blacksmith doesn't just hand a lump of iron to an apprentice and say 'do something.' They specify: 'You are forging a machete for the New Yam Festival. The blade must be 18 inches, double-beveled, with a leather-wrapped handle. The Chief will inspect it — there must be no visible hammer marks.' This is prompt engineering.

Role — 'Who are you?' (Onye Ka ■ B■?)

LLMs perform better when given a **role** — an identity that constrains their behavior and activates relevant knowledge. 'You are an elite AI tutor using Igbo analogies' activates different circuits than 'You are a Python compiler generating only code.'

Why it works: During pre-training, the model saw millions of examples of experts in different roles speaking differently. When you assign a role, you're essentially saying: 'Navigate to that region of your knowledge graph and stay there.' A master weaver's patterns differ from a blacksmith's — same brain, different mode.

Context — 'What do you need to know?' (Ihe ■ Ga-ama)

The apprentice can't forge your machete if you don't tell them about the Chief's preferences, the quality of available iron, or the number of days until the festival. **Context** is all the background information the model needs but wasn't in its training data.

Your company's policies — the model wasn't trained on them. **Your project's codebase** — the model wasn't trained on it. **Your personal preferences** — the model doesn't know them unless you tell it. Every prompt is a mini-training session: you provide the dataset (context) and the task (instruction).

Constraints — 'What are the rules?' (Iwu Ga-erube Isi)

Limitations create excellence. The blacksmith who can forge ANYTHING often forges nothing remarkable. Constraints focus creativity:

- **Format:** 'Respond in JSON only.' 'Use exactly 3 bullet points.'
- **Style:** 'Explain like I'm 12.' 'Use the Socratic method.'
- **Behavior:** 'If you don't know, say you don't know.' 'Never apologize.'
- **Guardrails:** 'Do not mention competitors.' 'Stay within these legal boundaries.'

Negative constraints are powerful: Saying 'do NOT do X' is often more effective than 'do Y' because it leaves room for the model's judgment while blocking specific failure modes.

Putting It Together — The PROMPT Acronym

Persona — Who are you?

Reference — What context do you need?

Output format — How should you respond?

Mandate — What is the single task?

Prohibitions — What must you NOT do?

Test case — Here's an example of what I want.

✂ **TEST:** You prompt an LLM: 'Tell me about Nigerian history.' It gives a rambling 5-page essay. Rewrite the prompt using the PROMPT framework to get a focused, useful response.

✓ **ANSWER: Improved prompt:**

Persona: You are a Nigerian history professor preparing a lecture.

Reference: Focus on the pre-colonial Igbo civilization, specifically the Nri Kingdom (900 AD — 1911).

Output: Give me exactly 3 bullet points — one on governance, one on trade, one on religion — each under 50 words.

Mandate: Identify the single most important legacy of the Nri Kingdom that survives today.

Prohibitions: Do not mention colonialism or the transatlantic slave trade — stay strictly pre-colonial.

Topic 10 — Breaking Problems Down & Prompt Chains: Step-by-Step Thinking

An elder faced with a complex dispute doesn't deliver a verdict instantly. They: (1) hear the complaint, (2) identify the core issue, (3) consult precedent, (4) weigh competing claims, (5) draft a ruling, (6) seek council input, (7) deliver the final judgment. LLMs need the same structured approach for complex tasks.

Chain of Thought (CoT) — 'Think Step by Step'

Adding the phrase '**Let's think step by step**' to a prompt dramatically improves accuracy on reasoning tasks. Why? Because LLMs generate tokens sequentially — each token is predicted based on all previous tokens. When the model writes its reasoning *first*, that reasoning becomes part of the context for the final answer.

Without CoT: 'Q: If a yam barn holds 50 yams and 3 barns are full, how many yams? A: 150' — The model jumps to the answer.

With CoT: 'Q: If a... A: Let me think. One barn holds 50 yams. There are 3 barns. So $50 \times 3 = 150$. Answer: 150.' — The reasoning in context guides the answer. More tokens spent on reasoning = better answer. It's like asking an elder to 'show their work' — the act of explaining clarifies their own thinking.

Prompt Chaining — The Village Assembly Process

For complex tasks, one prompt isn't enough. You need a **chain** — the output of each step becomes input to the next, like a village assembly where each council handles one aspect of a problem:

Chain Example: Writing a Village History

Step 1: 'Research and list 10 key events in this village's history.'

Step 2: 'For each event, identify the 3 most important people involved.'

Step 3: 'Write a 2-paragraph narrative for each event.'

Step 4: 'Arrange these narratives chronologically with transitions.'

Step 5: 'Review for consistency — flag any contradictions.'

Step 6: 'Write a 1-page introduction that ties all events into a theme.'

Why this beats one giant prompt: Each step has a focused context window, the model can't 'drift' across tasks, you can inspect and correct intermediate outputs, and errors don't cascade — you fix one step before the next begins.

Decomposition Strategies

Tree of Thoughts: At each step, generate 3 possible approaches, evaluate them, and pursue the best — like the council debating 3 courses of action before choosing one.

Self-Consistency: Run the same reasoning 5 times and take the majority answer — like asking 5 elders independently and going with the consensus.

Least-to-Most: Break the problem into sub-problems, solve the easiest first, then use that solution to tackle the harder ones — like an apprentice learning to forge nails before attempting a machete.

✍ **TEST:** You ask an LLM: 'Should our village invest in a new well or a new school?' Design a 4-step prompt chain that produces a rigorous recommendation.

✓ **ANSWER:** Step 1: 'List all stakeholders affected by each option (well vs school) and their interests.'

→ Output: list of stakeholders

Step 2: 'For each stakeholder, quantify the benefit (1-10 scale) of a well vs school. Consider health, education, economic impact.' → Matrix of scores

Step 3: 'Weigh the scores against our village's 5-year development plan. Identify which option aligns better.' → Alignment analysis

Step 4: 'Write a 3-paragraph recommendation with the analysis above. Address the losing side's concerns.' → Final recommendation

PHASE 5: AI Applications (Building Systems)

Topic 11 — APIs & Python Basics: Digital Messengers

In the old Igbo village, communication between clans required messengers — runners who carried carved message sticks (■f■) from one village to another. APIs are the digital messengers of the modern world.

What Is an API? (Ozi ■f■)

API = Application Programming Interface. It's a contract: 'If you send me a message in this specific format, I will respond in that specific format.' Like the message stick: the carvings had to follow a specific pattern (the API specification) for the receiving village to understand them.

REST API — The Village Market Protocol: The most common pattern. You send **requests** (GET = 'show me,' POST = 'create this,' PUT = 'update this,' DELETE = 'remove this') to **endpoints** (URLs like '/api/yams') and receive **responses** (usually JSON).

JSON — The Message Stick Carving: The universal format: `{'yam_variety': 'Ji-■cha', 'count': 150, 'harvested': true}`. Both human-readable and machine-parseable.

Calling an AI API — Sending a Message to the Oracle

When you 'call Claude' or 'call GPT,' you're sending an HTTP request to an API endpoint. Here's what's happening in Python:

Step 1: Prepare your message and API key.

Step 2: Send a POST request to the model's endpoint.

Step 3: Receive a JSON response containing the generated text.

Step 4: Extract and use the response.

It's identical to sending a messenger to consult the Dibia (diviner): you phrase your question (the prompt), the messenger carries it (the HTTP request), the diviner consults the spirits (the model), and the messenger returns with the answer (the response).

The Python Toolbox

requests — The messenger library. Sends HTTP requests.

openai / anthropic — AI-specific libraries that wrap the API calls.

json — Parse and create JSON responses.

os.getenv() — Never hard-code API keys. Read them from environment variables. Like the messenger who carries the ■f■ stick — the stick is recognized, but the messenger's identity is not written on it.

✍ **TEST:** Your village app needs to call 3 different LLMs (Claude, GPT, Gemini) and choose the best answer using majority voting. What Python pattern would you use to avoid duplicating the calling code 3 times?

✓ **ANSWER: A function with parameters.** Define `def ask_llm(model_name, api_key, prompt)` and call it 3 times with different model names. This is the DRY principle (Don't Repeat Yourself). Like a messenger who can deliver to any village — you just tell them which path to take, not rewrite the entire route each time.

Topic 12 — RAG & Vector Databases: The Yam Barn Memory System

An elder's wisdom is not all in their head. They consult the village archives (■ba Akw■kw■), stored in the yam barn — records of past harvests, dispute resolutions, and ancestral proverbs. LLMs need the same external memory system. This is **RAG — Retrieval-Augmented Generation**.

The Problem: LLMs Have No Long-Term Memory

Everything an LLM knows was frozen at training time. If your company hired a new CEO yesterday, the model doesn't know. If you wrote a new policy last week, the model hasn't read it. The context window is the only 'working memory,' and it's limited and expensive.

RAG solves this by giving the model a **searchable external memory**. Before answering your question, the system: (1) searches through all your documents, (2) finds the most relevant passages, (3) inserts them into the prompt as context, (4) the model answers based on this freshly retrieved information.

Vector Databases — The Yam Barn's Filing System

You can't just CTRL+F through 10,000 documents looking for 'yam harvest problems.' What if the document says 'crop failure' or 'tuber rot'? You need **semantic search** — finding documents by *meaning*, not keywords.

This is where **embeddings** (Topic 6) return. Every document is converted into a vector — a list of 768 or 1536 numbers that represents its *meaning*. These vectors are stored in a **vector database** (Pinecone, Weaviate, Qdrant, Chroma).

When you ask a question, it's also converted to a vector. The database finds documents whose vectors are **closest in meaning-space** — not closest in keyword matching, but closest in *what they're actually about*. It's like the yam barn organizer who doesn't just look for the word 'rotten' — they know to also pull records about 'soft spots,' 'dark patches,' and 'unusual odor.'

The RAG Pipeline — Building the Memory

Phase 1: Indexing (The Filing Day)

1. **Load:** Gather all documents (PDFs, web pages, databases).
2. **Chunk:** Split into manageable pieces (~500 tokens each). Like cutting a long scroll into pages.
3. **Embed:** Convert each chunk into a vector using an embedding model.
4. **Store:** Save vectors + original text in the vector database.

Phase 2: Retrieval (The Search)

1. **Embed the query:** Turn the user's question into a vector.

2. **Search:** Find the top-K most similar document chunks.
3. **Augment:** Insert retrieved chunks into the prompt as context.
4. **Generate:** The LLM answers using both its training + the retrieved docs.

Why RAG Is Revolutionary

- **Up-to-date knowledge:** Update documents, not the model.
- **Private data:** Your documents never entered the training set.
- **Source attribution:** You know *exactly* which document supported each claim.
- **Reduced hallucination:** The model is grounded in actual text, not just its memory.
- **Cost:** Cheaper than fine-tuning. No GPU training needed.

The Igbo wisdom: '*Onye na-eje ije na-anisi isi n'z.*' — The traveler who asks directions doesn't get lost. RAG lets the LLM 'ask directions' from your documents instead of relying on memory alone.

🔪 **TEST:** You're building a chatbot for Igbo customary law. The bot must answer questions based on 500 court rulings spanning 50 years. Why would RAG outperform just putting everything in a giant context window, and what's the ONE risk of RAG?

✓ **ANSWER: RAG wins** because: (1) 500 rulings likely exceed any context window, (2) irrelevant rulings won't confuse the model, (3) cost per query is lower (you only send relevant text, not all 500 documents).

The risk: *Retrieval failure.* If the embedding search fails to find the most relevant ruling (e.g., the query uses modern terms but the ruling uses archaic Igbo legal language), the model answers from incomplete context — and may hallucinate with confidence.

Topic 13 — Agents, Tools & Automation: Giving the AI Hands

The court poet (LLM) can compose beautiful verses — but they cannot plant yams, build barns, or negotiate trade. They have a mind but no body. **Agents** give the AI a body: tools, actions, and the ability to affect the world.

What Is an AI Agent? (Onye nwere Ike)

An **agent** is an LLM with a **loop**:

1. **Observe**: Read the current situation (user message, tool outputs).
2. **Think**: Reason about what to do next (the LLM's core ability).
3. **Act**: Call a tool (search web, run code, send email, click button).
4. **Observe**: Read the tool's output.
5. **Repeat** until the task is complete.

Unlike a simple chatbot that answers in one shot, an agent can take **multiple steps**, use **multiple tools**, and adapt its plan based on results. It's the difference between asking a historian 'tell me about the Nri Kingdom' and telling an assistant 'research the Nri Kingdom, write a 10-page report, email it to the council, and schedule a presentation for next Friday.'

Tools — The Agent's Hands

Web Search: 'Find the current market price of yams in Onitsha.' The agent searches the web, reads results, and reports.

Code Execution: 'Analyze this CSV of 10,000 harvest records and find trends.' The agent writes Python, runs it, reads the output.

API Calls: 'Send a payment of 50,000 to the palm oil supplier.' The agent calls a payment API with the correct parameters.

File Operations: 'Read all contracts in this folder and summarize each.' The agent opens files, reads, processes, writes summaries.

Desktop Control: 'Fill out this web form with the client's details.' The agent clicks, types, and submits — on the actual screen.

Each tool is an **action space** — a capability the agent can reason about and invoke. Like a master craftsman who can say 'hand me the chisel' instead of having to carve with their fingernails.

The Agent Loop — Planning and Replanning

An agent doesn't just execute a fixed plan. It **replans** based on results, like a village elder adjusting strategy based on new intelligence:

Plan: 'I will search the web for yam prices, then calculate the best deal.'

Act: Searches web. Finds that Onitsha market is closed for festival.

Replan: 'The Onitsha route failed. I will check Nnewi and Awka markets.'

Act: Searches those markets. Finds a deal in Awka.

Complete: 'The best yam price is in Awka at ₦120 per tuber.'

This loop — Plan → Act → Observe → Replan — is the **agentic pattern**. It's what separates a sophisticated assistant from a simple Q&A; bot.

Multi-Agent Systems — The Council of Specialists

One agent can do a lot. But multiple agents, each with different tools and expertise, can collaborate like a council of specialized elders:

- **Research Agent:** Only searches the web and reads documents
- **Coding Agent:** Only writes and runs code
- **Review Agent:** Only checks other agents' work for errors
- **Coordinator Agent:** Delegates tasks and merges results

The coordinator says: 'Research Agent, find market data. Coding Agent, build the analysis. Review Agent, verify the numbers.' They work in parallel and the coordinator assembles the final report.

The 7-Framework Questions — From Chatbot to Agent

Q1: What problem existed before agents?

A1: LLMs were purely conversational. They could tell you HOW to send an email, but couldn't actually send it. They were severed minds — wise but helpless.

Q2: Why did the chatbot-only paradigm fail for real work?

A2: Real work requires action: sending emails, querying databases, running code, controlling software. The chatbot could advise but never execute. Also, complex tasks require multiple steps with feedback — you can't 'summarize a 500-page document' in one prompt.

Q3: What design decision fixed it?

A3: The agent loop: give the LLM tools (functions it can call), let it observe results, and let it iterate. The LLM becomes the 'brain' of a system that has 'hands' — tools. Function-calling in the API layer enables this.

Q4: What sacrifice/trade-off?

A4: 1) **Latency:** Agents are slow — each tool call adds seconds. 2) **Cost:** Multiple LLM calls per task. 3) **Unpredictability:** An agent can go down wrong paths, use the wrong tool, or get stuck in loops. 4) **Safety:** An agent with real tools (email, payment, desktop) can cause real damage.

Q5: Where are the break points?

A5: 1) **Infinite loops:** Agent tries and fails the same action repeatedly. 2) **Tool misuse:** Agent calls tools with wrong parameters. 3) **Goal drift:** Agent forgets the original task across many steps. 4)

Context overflow: Tool outputs fill the context window. 5) **Permission boundaries:** How much autonomy is too much?

Q6: What hidden assumptions?

A6: 1) The LLM can accurately decide which tool to use and when. 2) Tool outputs are always trustworthy. 3) The agent can recover from any error. 4) The task is decomposable into tool-callable subtasks.

Q7: What evolved next?

A7: 1) **Guardrails and sandboxes** — limiting what agents can do. 2) **Human-in-the-loop** — requiring approval for high-risk actions. 3) **Hierarchical agents** — planners that delegate to workers. 4) **Long-term memory** — agents that remember past interactions. 5) **Self-critique** — agents that review their own outputs before acting.

🔪 **TEST:** Your agent is tasked with 'buy the cheapest flight to Lagos.' It searches, finds a \$85,000 flight, and autonomously purchases it. The user screams — they meant 'find and report,' not 'buy.' What critical agent design principle was violated?

✓ **ANSWER: Action confirmation / human-in-the-loop.** For irreversible or high-cost actions (payments, purchases, deletions, sending emails), the agent must *request confirmation* before executing. The principle: **read-only by default, write-with-permission**. Like a titled council member who can debate policy but cannot declare war without the Igwe's explicit approval.

PHASE 6: Deeper Understanding (The Roots)

Topic 14 — Linear Algebra & Calculus Concepts: The Hidden Physics

The blacksmith doesn't need to understand the crystalline structure of iron to forge a machete. But the master blacksmith who *does* understand it can forge blades that never break. This topic is the crystalline structure. You can use AI without it — but you'll never understand *why* things work or fail.

Vectors & Matrices — The Language of AI

Everything in AI is a **vector** (a list of numbers) or a **matrix** (a grid of numbers). A word embedding is a vector. An image is a matrix of pixel values. A neural network layer is a matrix multiplication.

Vector = A Path Through the Forest: [3, 4] means 'walk 3 steps east, 4 steps north.' A 768-dimensional embedding means 'walk 3 steps along dimension 1, 4 steps along dimension 2, ..., 0.7 steps along dimension 768.' Each dimension is a direction in meaning-space.

Dot Product = Similarity Score: How 'aligned' are two vectors? If two vectors point in the same direction, their dot product is large (similar meaning). If they point in opposite directions, it's negative (opposite meaning). If they're perpendicular, it's zero (unrelated). This is the engine behind attention scores, embedding similarity, and semantic search.

Matrix Multiplication = Transformation: A neural network layer is a matrix that transforms input vectors into output vectors. Like the blacksmith's anvil: iron (input) goes in one side, a different shape (output) emerges from the other. Every layer is a learned transformation matrix.

Calculus — The Science of Change (Mgbawwe)

Derivative = Rate of Change: 'If I increase the temperature of the forge by 1 degree, how much faster does the iron heat?' The derivative is the sensitivity of one thing to another. In AI, the derivative of the loss with respect to a weight tells us: 'If I tweak this weight up by 0.001, does the error go up or down, and by how much?'

Gradient = Direction of Steepest Ascent: A vector of partial derivatives — one for each weight in the network. It points in the direction that INCREASES the loss fastest. So we move in the OPPOSITE direction — gradient *descent*.

Chain Rule = Backpropagation's Engine: 'If A affects B, and B affects C, then how does A affect C?' Multiply the effects: $dC/dA = dC/dB \times dB/dA$. In a 100-layer network, the chain rule chains together 100 tiny effects to compute how the first layer's weight affects the final output's error.

Without the chain rule, deep learning would be impossible.

The Three Operations You Actually Need

1. **Matrix Multiply (`np.dot` / `@`):** Every layer does this. $\text{Input} \times \text{Weight Matrix} = \text{Output}$.
2. **Gradient (`autograd`):** PyTorch/TensorFlow compute this automatically. You never do calculus by hand — the framework traces every operation and applies the chain rule backward. It's automatic blacksmithing.
3. **Optimization Step (`optimizer.step()`):** Take the gradient, multiply by learning rate, subtract from current weights. One line of code. Repeat 100,000 times.

■ *Ilu Igbo: "Nke d■ n'elu b■ nke d■ n'ala. (As above, so below — patterns at one scale repeat at another.)"*

✂ **TEST: A neural network has 1,000,000 weights. After one forward pass, the loss is 2.3. Without autograd (automatic differentiation), how would you calculate the gradient for all 1,000,000 weights?**

✓ **ANSWER: Manually — and it would be nearly impossible.** Before autograd (pre-2010s), researchers derived gradients by hand for each network architecture, a painstaking and error-prone process. Autograd changed everything: the framework builds a computational graph of every operation (add, multiply, activate), then applies the chain rule automatically to compute all 1,000,000 gradients in one backward pass. This is why deep learning exploded — the calculus became a solved engineering problem, freeing researchers to focus on architecture design.

Topic 15 — Transformer Architecture Deep-Dive

We've arrived at the final topic — the architecture that powers every modern LLM. This is the blueprint of intelligence, seen through Igbo eyes.

The Full Transformer — A Village Council in Two Chambers

The Transformer has two main components, like an Igbo village with two governing bodies:

1. The Encoder (Ndị Nhazi — The Understanders): Reads and processes the input text. Builds a rich, contextual representation of every word. 'What does this sentence mean, in full context?' — Used by BERT, embedding models.

2. The Decoder (Ndị Mepụtara — The Generators): Generates the output text, one token at a time. At each step, it attends to both the encoded input AND its own previous outputs. 'Given what I've understood, and what I've said so far, what should I say next?' — Used by GPT, Claude, all generative LLMs.

GPT is Decoder-Only: Modern LLMs dropped the encoder entirely. They just generate. At each step, the decoder attends to all previous tokens it has generated (self-attention) — no separate encoder needed.

The Transformer Block — Repeated Wisdom Layers

A Transformer isn't one big neural net. It's a **stack** of identical blocks, each containing the same components, repeated 12, 24, 96, or more times. Each block is like one round of the council meeting:

Component 1 — Multi-Head Self-Attention: Every token looks at every other token, weighs relevance, and aggregates context. (Topic 7's deep dive.)

Component 2 — Feed-Forward Network (FFN): After gathering context from all other tokens, each token processes that information through a small neural network (2 layers, usually). This is where the 'thinking' happens — the attention gathered the raw materials; the FFN shapes them into understanding.

Component 3 — Layer Normalization (LayerNorm): Keeps the numbers stable. Without it, values would explode (huge numbers) or vanish (near zero) across 96 layers. Like the council moderator who ensures no single elder dominates the discussion and no voice is completely silenced.

Component 4 — Residual Connections (Skip Connections): The input to each sub-layer is ADDED to its output. 'Here's what I knew before + here's what I learned now.' This prevents

information loss across deep stacks — like an elder who says 'I've heard the new testimony, but I also remember the original oath.'

The Full Token Journey

Let's follow one token — the word 'akw■kw■' — through a 12-layer decoder (simplified GPT):

- 1. Tokenization:** 'akw■kw■' → Token ID 18472
- 2. Embedding:** Token ID 18472 → Vector of 768 numbers
- 3. Positional Encoding:** 'You are the 47th token in this sequence.' Adds position information to the embedding vector.
- 4. Block 1 — Attention:** 'akw■kw■' looks at all previous tokens. High attention on 'ag■' (read) and 'n■■■' (in the house). Context: 'reading-related, indoors' → first layer of meaning.
- 5. Block 1 — FFN:** Processes the attended information. Preliminary understanding forms.
- 6. Blocks 2-11:** Each block refines the meaning. Block 4 might resolve syntactic role (subject vs object). Block 8 might resolve semantic ambiguity (book vs leaf). Block 11 might prepare for generation (this word will influence the next token).
- 7. Block 12 — Final Layer:** Produces the final hidden state for 'akw■kw■.'
- 8. Output Projection:** Multiply by the output matrix → vector of size 50,000 (vocabulary size).
- 9. Softmax:** Convert to probabilities.
- 10. Sample:** Pick the next token probabilistically. The cycle repeats.

The Numbers (GPT-3 as Reference)

Layers: 96

Attention heads per layer: 96

Embedding dimension: 12,288

Total parameters: 175 billion

Training tokens: ~300 billion

Training cost: Estimated \$4.6M in compute

That's 96 rounds of council meeting, with 96 specialized elders per round, each tracking different relationships, processing 12,288-dimensional representations of every word. And it does this for every single token it generates — in milliseconds. This is the machine you're learning to wield, Builder.

The 7-Framework Questions — Why Transformers Won

Q1: What problem did Transformers solve that RNNs/LSTMs couldn't?

A1: Long-range dependencies and parallelization. RNNs processed tokens sequentially and their memory of early tokens degraded. Transformers process all tokens simultaneously with perfect long-range access via attention.

Q2: Why did RNNs fail at scale?

A2: 1) Vanishing/exploding gradients made training deep RNNs unstable. 2) Sequential processing meant you couldn't parallelize across GPUs. Training on 300B tokens sequentially would take years.

Q3: What design decisions made Transformers work?

A3: 1) Pure attention — no recurrence. 2) Positional encoding — baked order into otherwise order-blind attention. 3) Multi-head attention — multiple relationship types in parallel. 4) Residual connections + LayerNorm — stable training at unprecedented depth.

Q4: What trade-offs?

A4: 1) $O(n^2)$ complexity — attention is quadratic in sequence length. 2) No inherent memory beyond the context window. 3) Fixed computational budget per layer regardless of token importance. 4) Positional encodings impose a maximum length.

Q5: Where are the break points?

A5: 1) **Context length:** Beyond ~100K tokens, the quadratic cost becomes prohibitive. 2) **Attention collapse:** In very deep networks, attention can become uniform (all tokens get equal weight — useless). 3) **Training instability:** 175B parameters are hard to train; loss spikes can ruin a \$4M training run. 4) **Inference cost:** Every generated token requires a full forward pass through all layers.

Q6: What hidden assumptions?

A6: 1) Position matters in a fixed, linear way (positional encoding assumption). 2) Softmax attention (summing to 1) is the right way to distribute focus. 3) Every token-pair interaction is equally worth computing (most aren't). 4) Larger models + more data always improves (scaling laws hold).

Q7: What is evolving next?

A7: 1) **Linear attention / State-space models** (Mamba, RWKV) — $O(n)$ instead of $O(n^2)$. 2) **Mixture of Experts (MoE)** — only activate relevant parameters per token, not all 175B. 3) **Speculative decoding** — use a small model to draft tokens, verify with the big model. 4) **Infinite context** — compression, external memory, ring attention. 5) **Multimodal Transformers** — same architecture processing text, images, audio, and video simultaneously.

🔪 **TEST: GPT uses a decoder-only architecture. BERT uses an encoder-only architecture. What can BERT do that GPT fundamentally cannot, and what can GPT do that BERT cannot?**

✓ **ANSWER: BERT (encoder-only):** Can look at ALL tokens simultaneously (bidirectional) because it's not generating — it's understanding. Perfect for classification, sentiment analysis, named entity recognition. It sees the full sentence at once.

GPT (decoder-only): Can *generate* text, because it's trained to predict the next token. It uses causal (masked) attention — each token only sees PREVIOUS tokens, not future ones. This allows autoregressive generation: predict token 1, then token 2 given token 1, then token 3 given 1 and 2, etc.

The trade-off is the Igbo proverb: 'Onye na-ele anya n'ihu anaghị ahụ azụ.' (One who looks forward cannot see behind.) GPT looks forward (generates) but can't see future tokens. BERT sees everything at once but can't generate.

You Have Completed the Builder's Path

Fifteen topics. Six phases. You've traveled from the village council to the 96-layer Transformer, from rules to agents, from yam barns to vector databases. You now possess the 80/20 knowledge — the 20% of concepts that explain 80% of everything in AI.

What you can now do:

- Read AI papers and understand the architecture diagrams
- Debug why a model is failing (loss, attention, embeddings, context)
- Build RAG systems, prompt chains, and agents
- Have informed conversations with ML engineers
- Know what you don't know — and where to go deeper

Where to go from here:

- **Build:** The only way to solidify this is to build something. Start with a simple RAG chatbot. Then an agent with tools.
- **Read:** 'Attention Is All You Need' (the Transformer paper). 'Deep Learning' by Goodfellow, Bengio, Courville.
- **Practice:** HuggingFace tutorials. Fast.ai course. Build a fine-tuned model on your own data.
- **Go deeper:** The optional topics you skipped — the full linear algebra, the calculus, the system design of training pipelines.

" Onye kwe, chi ya ekwe. — If a person agrees, their chi (personal god/spirit) also agrees. The Builder's Path begins with agreement to learn. You have agreed. Now build. "

— End of the Builder's Path —